

Seaborn Exercises

- This homework is designed to test your skills at creating data visualizations using Seaborn.
- There are **FOUR** tasks. Solve all four.
- You are highly encouraged to refer to the official Seaborn documentation (apart from class notes) while solving this homework.
- Link to Seaborn documentation: <https://seaborn.pydata.org/>
- Additional resources: Matplotlib tutorial in Sec. 1.4 in the Scientific Python lectures (link available on the course website under "Syllabus and General Information")

Imports

Run the cell below to import the libraries

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns

import matplotlib.pyplot as plt
```

The Data

The dataset for this homework is also available on Kaggle:

<https://www.kaggle.com/rikdifos/credit-card-approval-prediction>

Brief Description:

- Credit score cards are a common risk control method in the financial industry.
- It uses personal information and data submitted by credit card applicants to predict the probability of future defaults and credit card borrowings.
- The bank is able to decide whether to issue a credit card to the applicant.
- Essentially, credit scores can help quantify the magnitude of risk.

Feature Information:

application_record.csv

Feature name	Explanation	Remarks
ID	Client number	
CODE_GENDER	Gender	

application_record.csv

FLAG_OWN_CAR	Is there a car	
FLAG_OWN_REALTY	Is there a property	
CNT_CHILDREN	Number of children	
AMT_INCOME_TOTAL	Annual income	
NAME_INCOME_TYPE	Income category	
NAME_EDUCATION_TYPE	Education level	
NAME_FAMILY_STATUS	Marital status	
NAME_HOUSING_TYPE	Way of living	
DAYS_BIRTH	Birthday	Count backwards from current day (0), -1 means yesterday
DAYS_EMPLOYED	Start date of employment	Count backwards from current day(0). If positive, it means the person currently unemployed.
FLAG_MOBIL	Is there a mobile phone	
FLAG_WORK_PHONE	Is there a work phone	
FLAG_PHONE	Is there a phone	
FLAG_EMAIL	Is there an email	
OCCUPATION_TYPE	Occupation	
CNT_FAM_MEMBERS	Family size	

```
In [2]: df = pd.read_csv('application_record.csv')
```

```
In [3]: df.head(3)
```

```
Out[3]:
```

	ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	
0	5008804	M	Y	Y	0	
1	5008805	M	Y	Y	0	
2	5008806	M	Y	Y	0	

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 438557 entries, 0 to 438556
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                     438557 non-null  int64
1   CODE_GENDER                           438557 non-null  object
2   FLAG_OWN_CAR                           438557 non-null  object
3   FLAG_OWN_REALTY                        438557 non-null  object
4   CNT_CHILDREN                           438557 non-null  int64
5   AMT_INCOME_TOTAL                       438557 non-null  float64
6   NAME_INCOME_TYPE                       438557 non-null  object
7   NAME_EDUCATION_TYPE                    438557 non-null  object
8   NAME_FAMILY_STATUS                     438557 non-null  object
9   NAME_HOUSING_TYPE                      438557 non-null  object
10  DAYS_BIRTH                             438557 non-null  int64
11  DAYS_EMPLOYED                           438557 non-null  int64
12  FLAG_MOBIL                             438557 non-null  int64
13  FLAG_WORK_PHONE                         438557 non-null  int64
14  FLAG_PHONE                             438557 non-null  int64
15  FLAG_EMAIL                             438557 non-null  int64
16  OCCUPATION_TYPE                         304354 non-null  object
17  CNT_FAM_MEMBERS                         438557 non-null  float64
dtypes: float64(2), int64(8), object(8)
memory usage: 60.2+ MB
```

```
In [5]: df.describe()
```

```
Out[5]:
```

	ID	CNT_CHILDREN	AMT_INCOME_TOTAL	DAYS_BIRTH	DAYS_EN
count	4.385570e+05	438557.000000	4.385570e+05	438557.000000	43855
mean	6.022176e+06	0.427390	1.875243e+05	-15997.904649	6056
std	5.716370e+05	0.724882	1.100869e+05	4185.030007	13876
min	5.008804e+06	0.000000	2.610000e+04	-25201.000000	-1753
25%	5.609375e+06	0.000000	1.215000e+05	-19483.000000	-310
50%	6.047745e+06	0.000000	1.607805e+05	-15630.000000	-146
75%	6.456971e+06	1.000000	2.250000e+05	-12514.000000	-37
max	7.999952e+06	19.000000	6.750000e+06	-7489.000000	36524

TASKS

Recreate the plots shown in the markdown image cells.

Each plot also contains a brief description of what it is trying to convey.

Note, these are meant to be quite challenging. Start by first replicating the most basic form of the plot, then attempt to adjust its styling and parameters to match the given image.

Closer reproductions will receive higher scores.

NOTE: You may need to perform extra calculations on the Pandas DataFrame before calling Seaborn to create the plot.

TASK: Recreate the Scatter Plot shown below

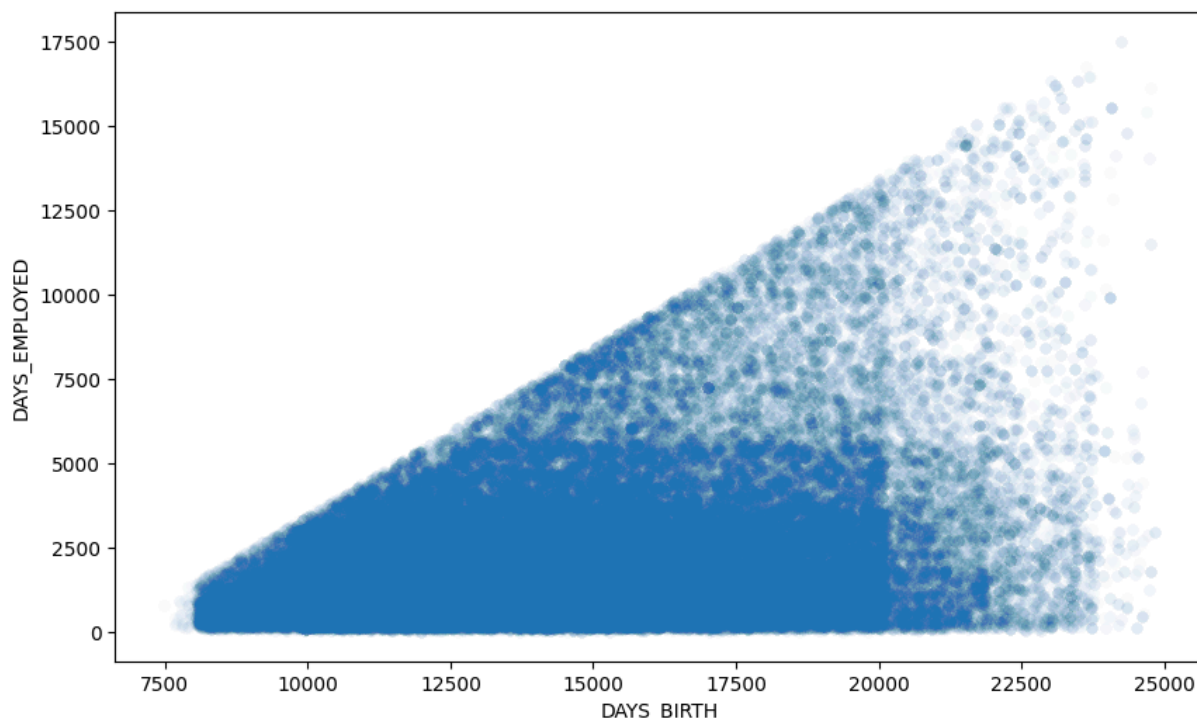
- The scatterplot attempts to show the relationship between the days employed versus the age of the person (DAYS_BIRTH) for people who were not unemployed.
- Note, to reproduce this chart you must **remove unemployed people** from the dataset first.
- Also note the **sign** of the axis, they are both transformed to be positive.
- What do the `alpha` and `linewidth` parameters in a Seaborn scatterplot control? Since there are so many points stacked on top of each other, feel free to modify these two parameters to present a clear plot.

 No description has been provided for this image

```
In [37]: # CODE HERE TO RECREATE THE PLOT SHOWN ABOVE
# Remove unemployed people from the dataset
task_one_df = df[df['DAYS_EMPLOYED'] < 0].copy()

# Transform the sign of the axis to be positive
task_one_df.loc[:, 'DAYS_BIRTH'] = task_one_df['DAYS_BIRTH'].abs()
task_one_df.loc[:, 'DAYS_EMPLOYED'] = task_one_df['DAYS_EMPLOYED'].abs()

# Create the scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(data=task_one_df, x='DAYS_BIRTH', y='DAYS_EMPLOYED', alpha=0.5, linewidth=1)
plt.show()
```



TASK: Recreate the Distribution Plot shown below:

 No description has been provided for this image

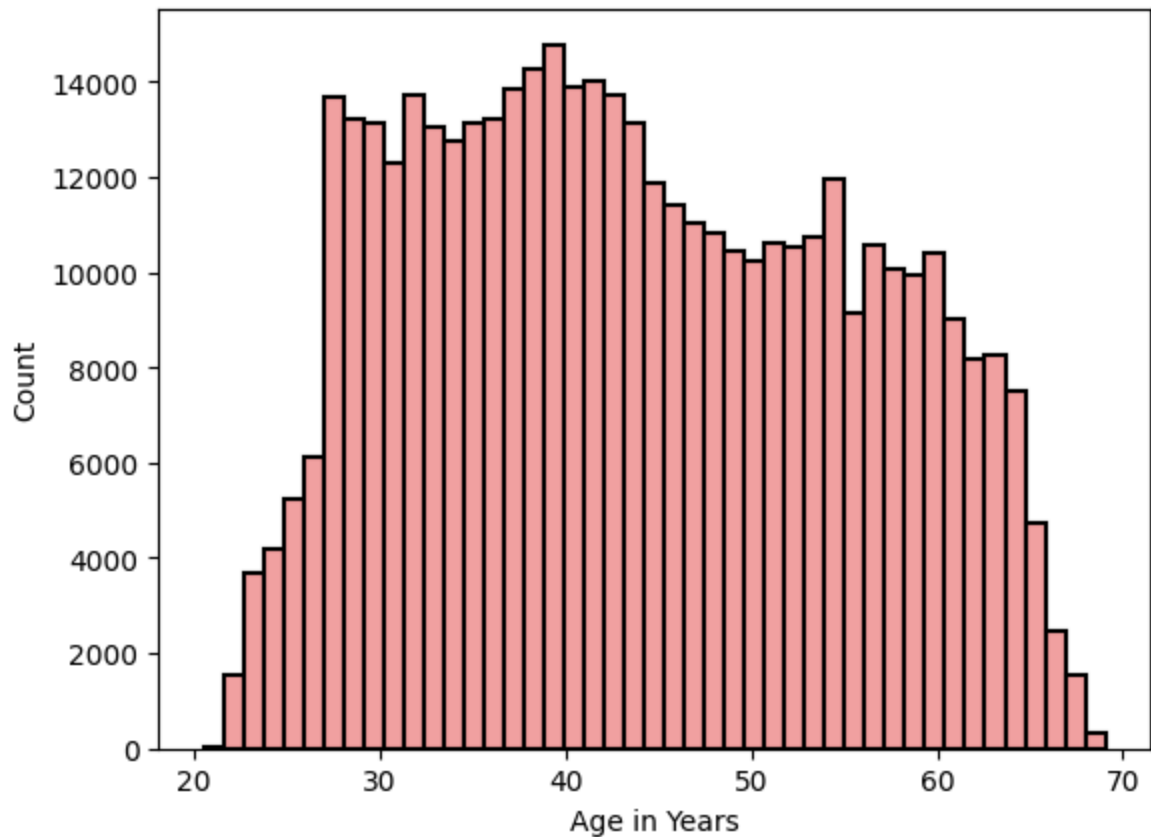
Note, you will need to figure out how to calculate "Age in Years" from one of the columns in the DF. Think carefully about this.

```
In [ ]: # CODE HERE TO RECREATE THE PLOT SHOWN ABOVE
task_two_df = df.copy()

# Transform the sign of the axis to be positive
task_two_df.loc[:, 'DAYS_BIRTH'] = task_two_df['DAYS_BIRTH'].abs()

# Calculate age in years
task_two_df['AGE_YEARS'] = task_two_df['DAYS_BIRTH'] / 365

# Create the histogram
plt.figure()
sns.histplot(task_two_df['AGE_YEARS'], color='lightcoral', bins=45, linewidth=1)
plt.xlabel('Age in Years')
plt.ylabel('Count')
plt.show()
```



TASK: Recreate the Categorical Plot shown below:

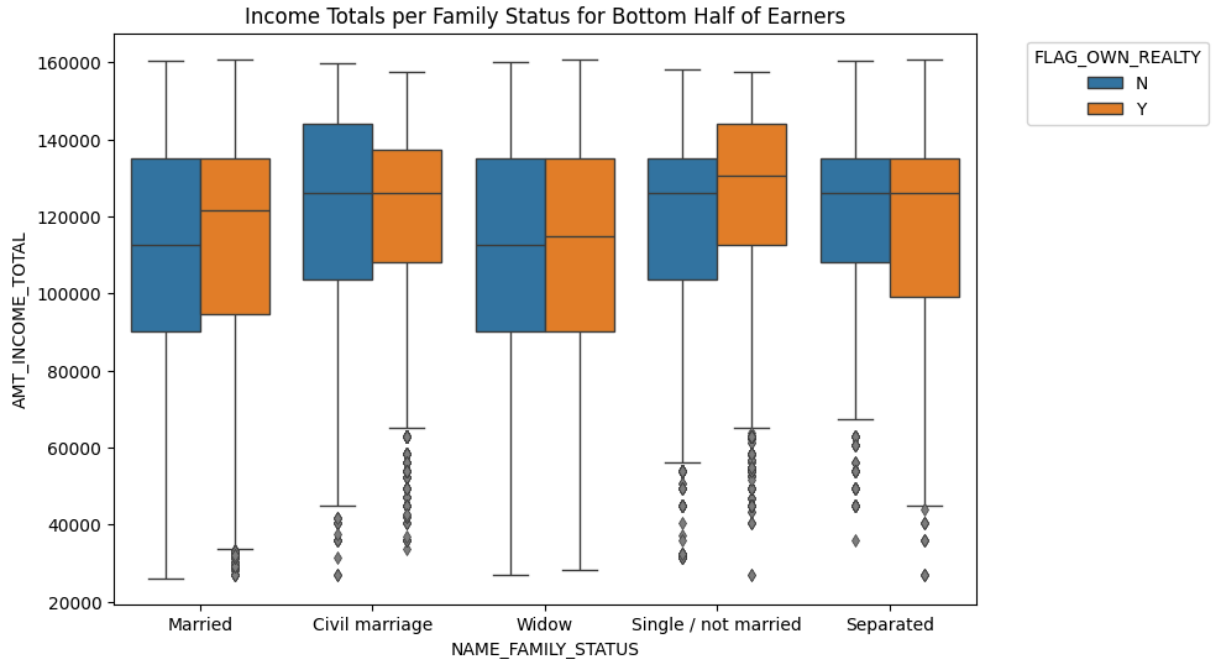
No description has been provided for this image

- This plot shows information only for the **bottom half** of income earners in the data set.
- It shows the boxplots for each category of NAME_FAMILY_STATUS column for displaying their distribution of their total income.
- Note: You will need to adjust or only take part of the dataframe *before* recreating this plot.
- You may want to explore the `order` parameter to get the xticks in the exact order shown here

```
In [76]: # Code here
# Filter the bottom half of income earners
task_three_df = df[df['AMT_INCOME_TOTAL'] <= df['AMT_INCOME_TOTAL'].median()]

# Create the categorical plot
plt.figure(figsize=(9, 6))
sns.boxplot(data=task_three_df, x='NAME_FAMILY_STATUS', y='AMT_INCOME_TOTAL',
            order=['Married', 'Civil marriage', 'Widow', 'Single / not married', 'Separated'],
            flierprops=dict(marker='d', markerfacecolor='gray', markersize=5, marker
plt.legend(title='FLAG_OWN_REALTY', bbox_to_anchor=(1.05, 1), loc='upper left')
```

```
plt.title('Income Totals per Family Status for Bottom Half of Earners')
plt.show()
```



Heatmaps

In Seaborn, **heatmaps** are used to **visualize data in a matrix format**, where the individual values in the matrix are represented as **colored cells**. Heatmaps are especially useful for showing the magnitude of data and patterns across a 2D space.

Each cell in the heatmap is colored based on the value it contains, with a color gradient that reflects the intensity or magnitude of the value.

Seaborn Heatmap documentation:

<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

What Heatmaps Show:

1. **Matrix of values:** Heatmaps visualize data that is organized in a grid (e.g., a correlation matrix, confusion matrix, or any two-dimensional data).
2. **Color-coded magnitude:** The color of each cell in the heatmap reflects the **magnitude** of the corresponding data point. Lighter or darker colors represent higher or lower values, depending on the color map.
3. **Patterns and relationships:** Heatmaps are excellent for identifying **patterns**, **trends**, or **correlations** between variables by using colors to highlight significant values or clusters in the data.

Common Use Cases for Heatmaps:

- **Correlation matrix:** A heatmap is often used to visualize a **correlation matrix**, where each cell represents the correlation coefficient between two variables.
- **Confusion matrix:** It is also commonly used to display a **confusion matrix** in classification tasks, where the color shows how often different categories are confused with each other. (We will see this later in the course)
- **Clustered data:** Heatmaps can highlight patterns in clustered data, making it easier to see relationships between features or groups.

TASK: Recreate the Heat Map shown below:

 No description has been provided for this image

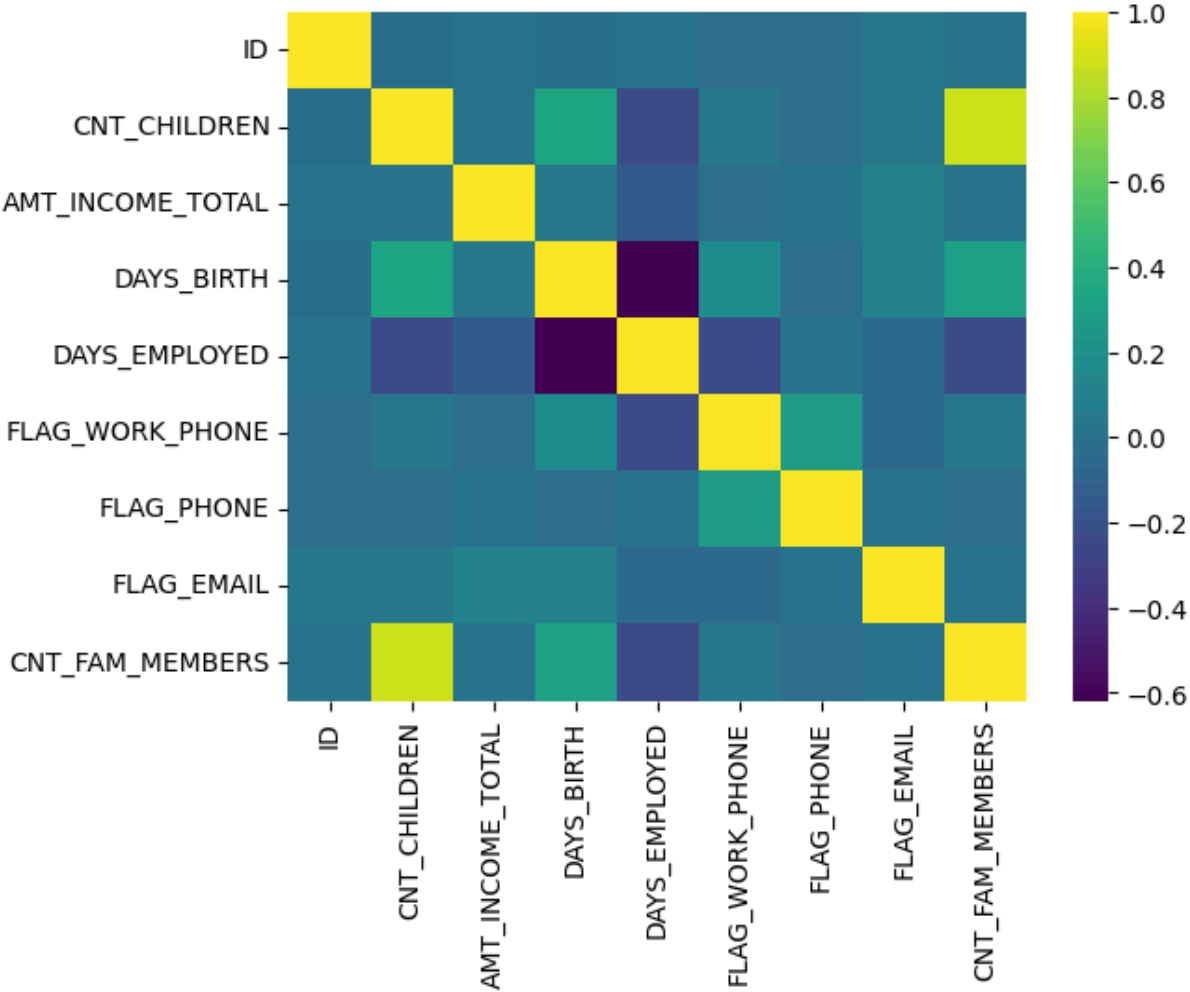
- This heatmap shows the correlation between the columns in the dataframe. You can get correlation with `.corr()`
- Note here, that the `FLAG_MOBIL` column has `NaN` correlation with every other column, so you should drop it before calling `.corr()`.
- Finally, what `cmap` value will give you this colormap? Choose from the Matplotlib color palette [available here](#).

```
In [84]: # CODE HERE
# Drop the FLAG_MOBIL column
task_four_df = df.drop(columns=['FLAG_MOBIL'])

# Select only numerical columns
task_four_df = task_four_df.select_dtypes(include=[np.number])

# Calculate the correlation matrix
corr_matrix = task_four_df.corr()

# Create the heatmap
sns.heatmap(corr_matrix, annot=False, cmap='viridis')
plt.show()
```

```
In [ ]:
In [ ]:
In [ ]:
```